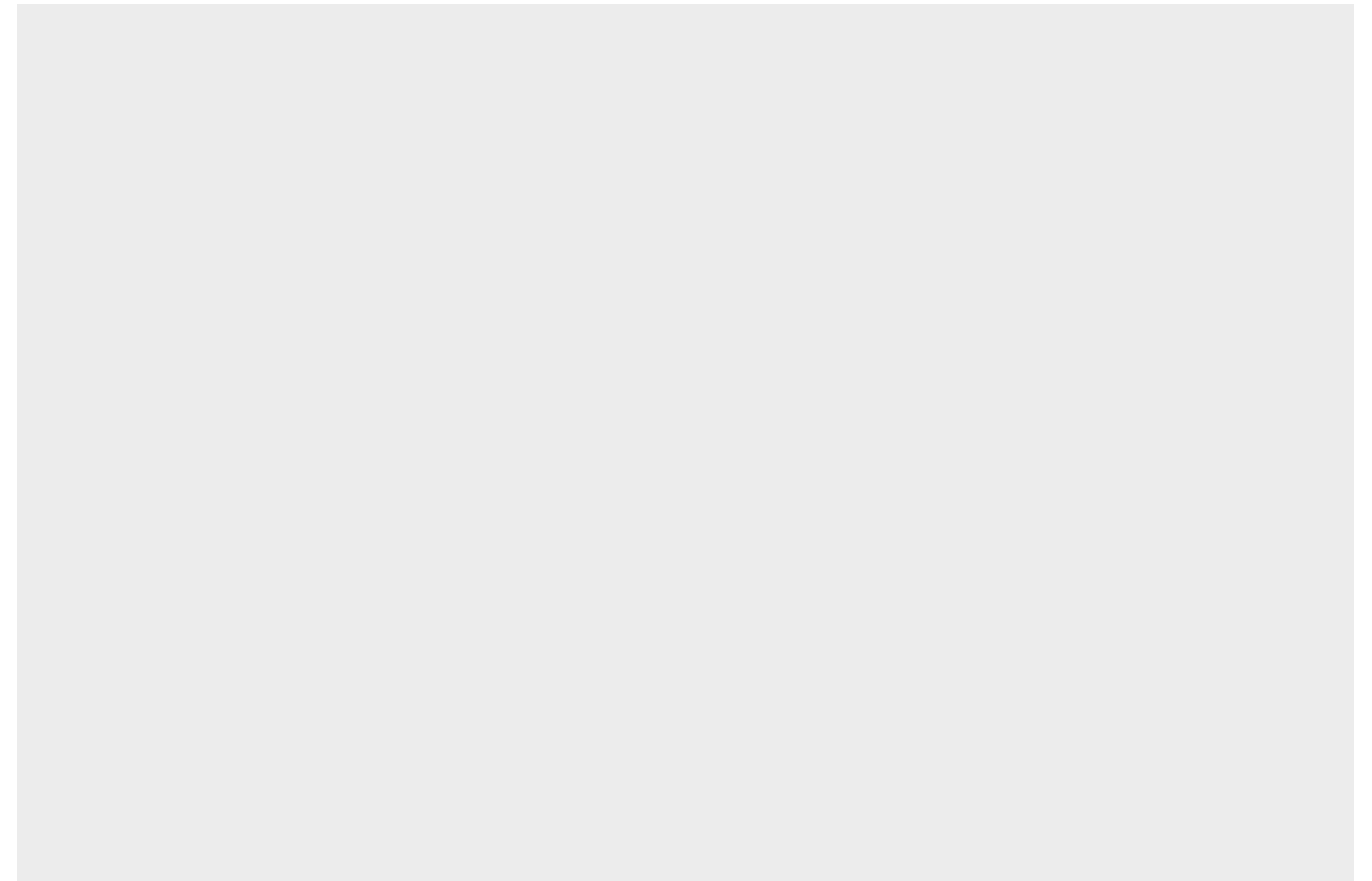
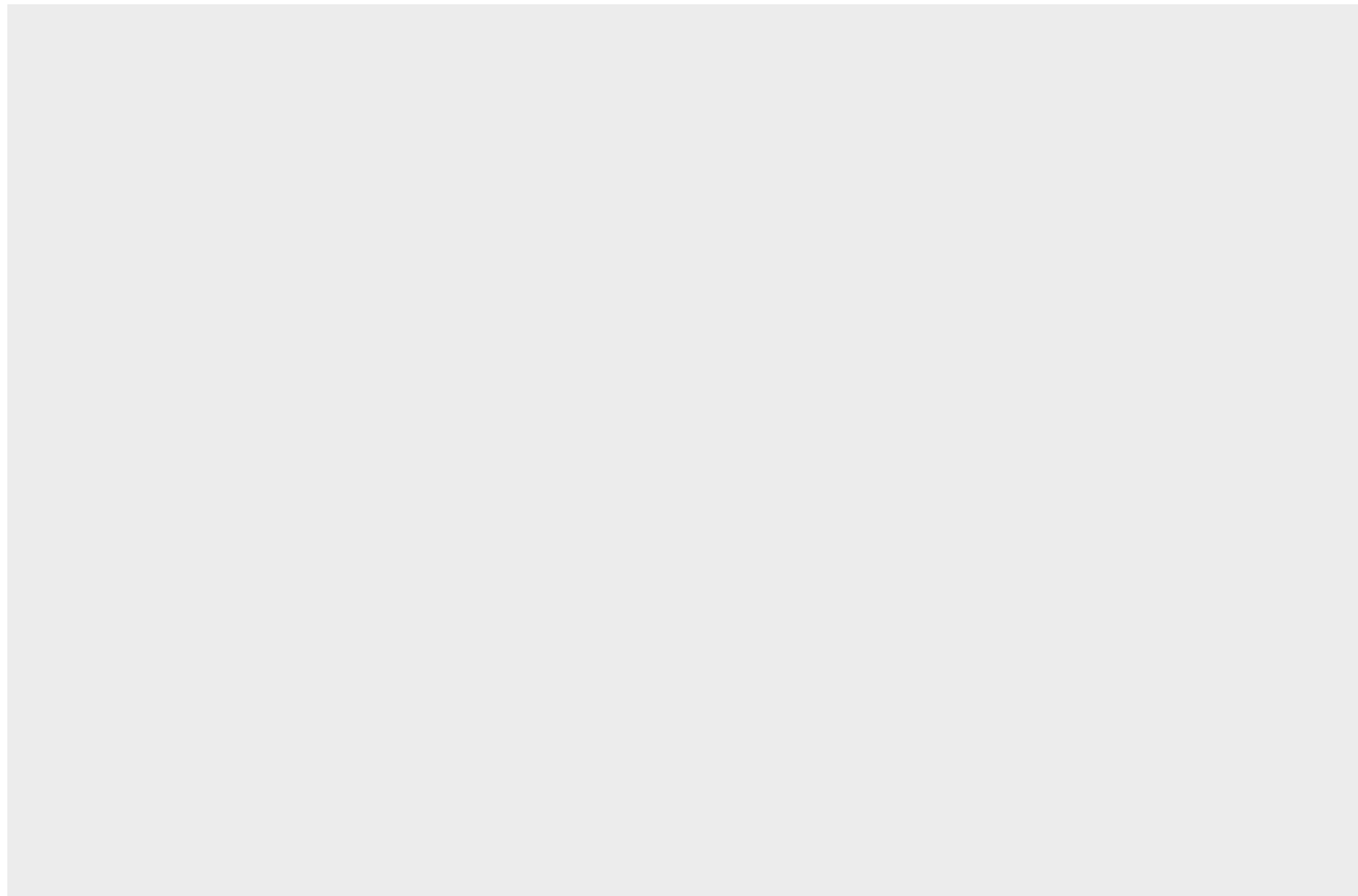
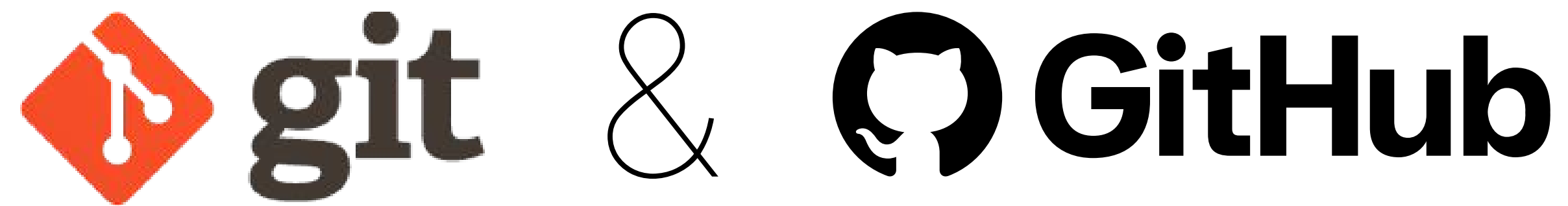


What is git?

What is GitHub?





What is git?

Git is a free and open source **distributed version control system** for projects of any size. Git enables **version tracking, branching,** and seamless **teamwork**.

What is GitHub?

GitHub is a **cloud platform** for git to **host, review, and share** projects with ease. GitHub adds cloud hosting, pull requests, issues, and collaboration tools.

Feature	Git	GitHub
Type	Version control system (VCS)	Web based Hosting Service
Function	Tracks and manages code changes locally	Stores Git repositories in the cloud and enables collaboration
Installation	Must be installed on your system	Accessible through a web browser (no installation needed)
Usage	Used for local version control and branching	Used for sharing, reviewing, and managing Git projects online
Collaboration	Limited to local or manual sharing	Real-time collaboration with teams and contributors
Interface	Command-line tool (CLI)	Web-based UI and also supports Git CLI
Main Purpose	Track code history and manage changes locally	Centralized hub to host, view, and contribute to Git projects
Offline Support	Fully functional offline	Requires internet to access remote features

Common git concepts:

Repository

Clone

Stage

Commit

Branch

Merge

Pull

Push

Common git concepts:

Repository: a directory where git monitors your project files and records their revision history

Clone: creates a local copy of a remote repository on your machine

Stage: specific changes that you want git to include in the next snapshot

Commit: records the staged changes as a permanent version in the project history

Common git concepts:

Branch: lets you develop new features or experiment without affecting the main project

Merge: integrates changes from one branch into another

Pull: fetches and applies updates from a remote repository to your local one

Push: uploads your local commits to a remote repository

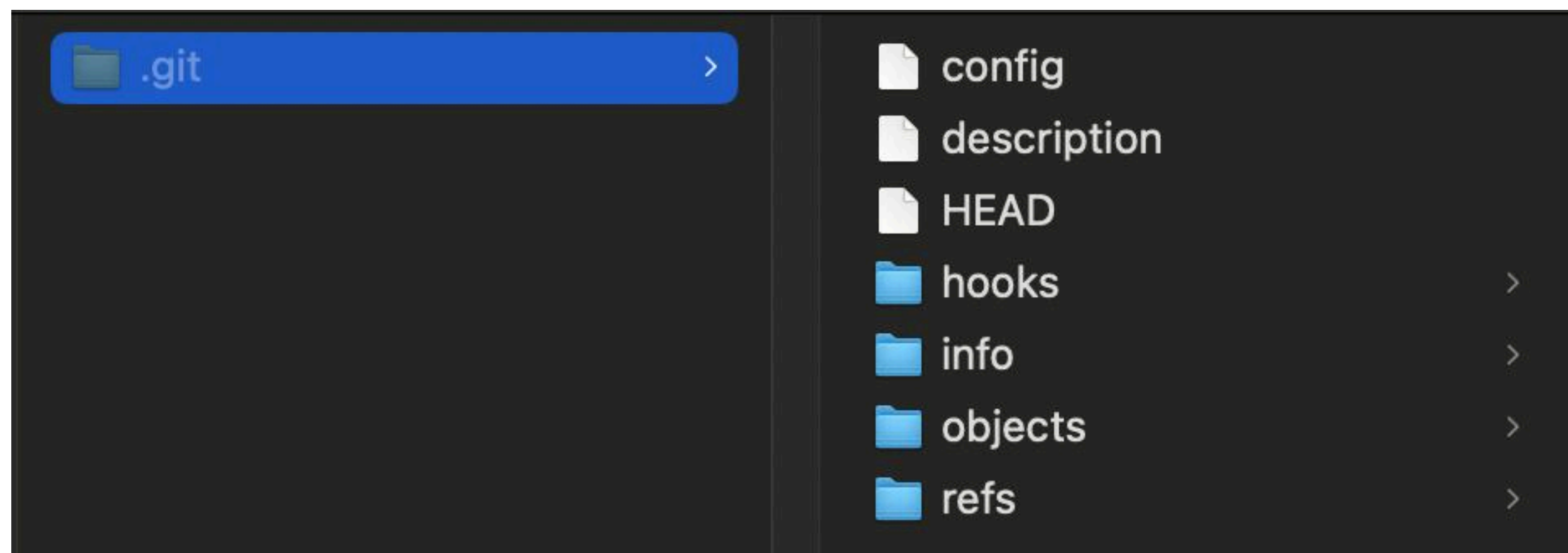
Git repository structure:

- 1. Working directory:** your local directory where you make the project (write code) and make changes to it
- 2. Staging area:** an area where you first need to put your project before committing (for instance, for code review)
- 3. Local repository:** your local repository where you commit changes to the project before pushing them to the central repository on GitHub. This corresponds to the .git folder
- 4. Central repository:** the main project on the server, a copy of which is with every team member as a local repository

Git commands – starting:

New project

1. Go to your project's folder
2. ***git init***
3. Now you have your .git directory prepared:



Connect to a remote repository

1. Go to your project's folder
2. ***git remote add***
<reponame> <repourl>
3. Traditionally the main remote git repository is called **origin**

Git commands – staging:

1. You're in your project directory
2. ***git add <filename>*** // to add one specific file
or
git add . // to add all untracked files and unstaged changes
3. Now you can check what you added: ***git status***
4. Made a mistake? You can undo your staging:
git reset <filename> // to unstage one specific file
or
git reset // to unstage everything

Git commands – committing:

1. You're in your project directory
2. ***git commit -m 'message'*** // commit your staged files
or
git commit -am 'message' // stage + commit your changes
3. What to write in the 'message'?
One commit – one change. Briefly describe what was done:
<type>(optional-scope): <short summary>
For example: feat(auth): add OAuth2 authentication
For more info: <https://www.conventionalcommits.org/>

Git commands – pushing and pulling:

Push changes

1. Go to your project's folder

2. ***git push***

// to push the current
branch to its remote
"tracking branch"

or

git push origin main

// to push your main branch
to the remote origin

Pull changes

1. Go to your project's folder

2. ***git pull***

// to fetch changes and
then merge them into your
current branch

or

git pull origin main

Git commands – working with branches:

1. List of branches you have:

git branch

2. Create a branch:

git branch <branchname> // without switching

or

git switch -c <branchname> // create and switch to it

3. Switch to a branch:

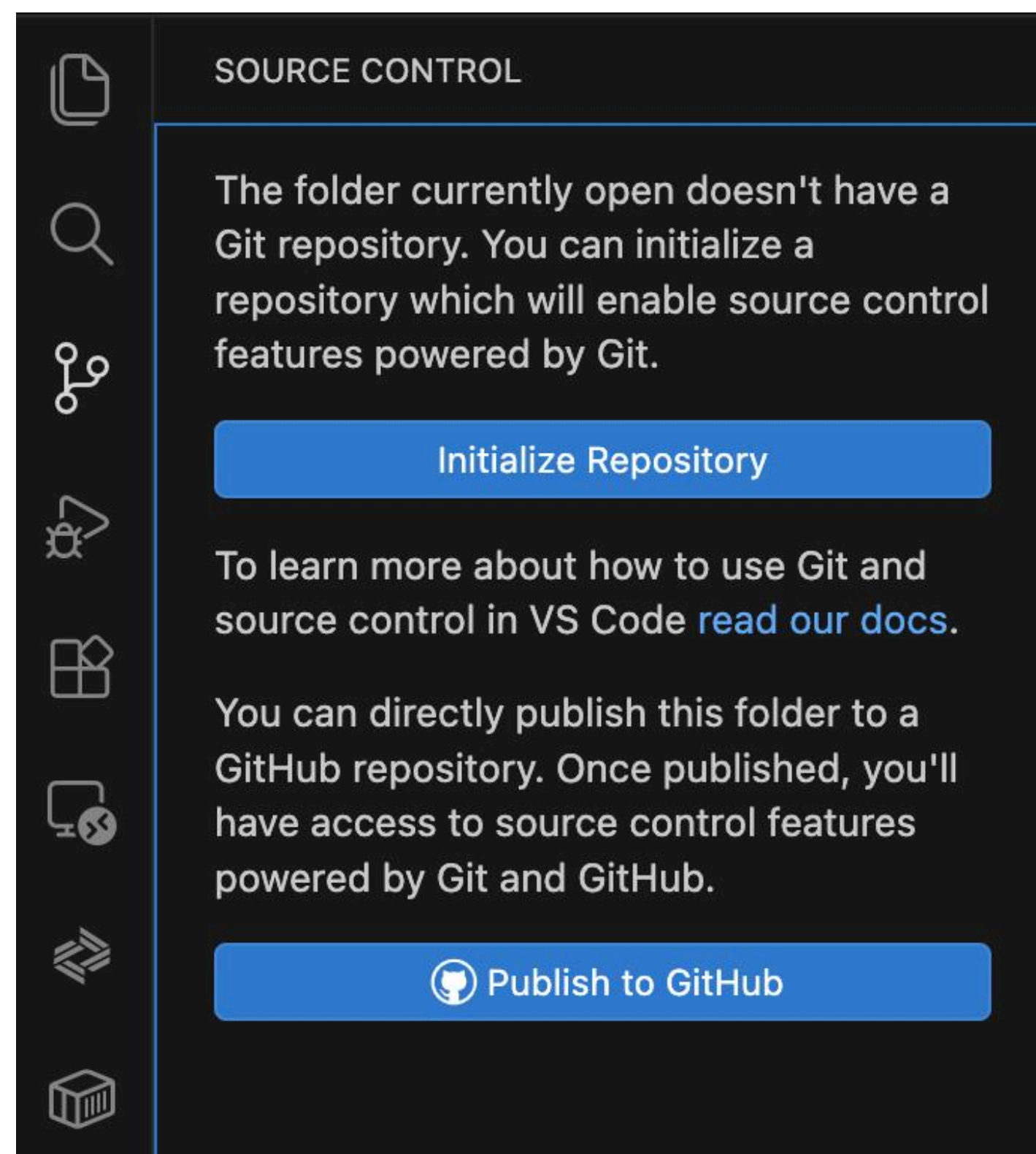
git switch <branchname>

Git commands – merging:

1. Choose your master branch
2. ***git merge <branchname>*** // merge with <branchname>
3. Always commit before starting a merge
4. Regularly merge from the main branch into your feature branch to minimize conflicts
5. Read and resolve conflicts carefully-don't just accept all changes blindly

P.S.

1. Configure your IDE for your git repository:



2. Read the documentation to further hone your skills:
[https://git-scm.com/docs/
git#_git_commands](https://git-scm.com/docs/git#_git_commands)
<https://docs.github.com/en>

3. Learn the markdown language to improve your *.md files: [https://
www.markdownguide.org/](https://www.markdownguide.org/)